databricks

amadeus

OVERVIEW

AI and Data Science on Databricks

Databricks platform architecture

1 DATA STORAGE AND MOVEMENT

- Table formats
- Ingestion and pipelines
- Transactional and real-time data
- Table maintenance

2 UNITY CATALOG

- Catalog and access
- Policy and classification
- Observability
- Cost and usage
- Semantic definitions
- Data sharing

3 DATA SCIENCE AND MACHINE LEARNING

- Workspace
- Feature engineering in Unity Catalog
- Training compute
- MLflow 3
- Model Serving
- Deployment automation

4 AGENT DEVELOPMENT

- Model access
- Custom LLMs
- Batch inference: AI Functions
- Knowledge retrieval
- Tools
- Agent build
- Agent deployment
- Agent memory
- No-code agents: Genie Agents

5 OBSERVABILITY AND EVALUATION

- MLflow Tracing
- MLflow evaluation
- Prompt Registry
- Runtime telemetry

6 AI GOVERNANCE AND SECURITY

- Unity Catalog for AI assets
- Unity Gateway: policies
- Unity Gateway: monitoring
- Unity Gateway: cost and capacity
- Unity Gateway: coding agent governance
- Security and compliance

7 AI FOR BUSINESS USERS

- Genie One
- Genie Agents
- Genie Ontology
- Genie integrations
- AI/BI Dashboards
- Applications
- Genie ZeroOps
- Genie budgets

8 DEVELOPER TOOLS

- Omnigent
- Agentic code converter

9 VERTICAL APPLICATIONS

- CustomerLake
- Lakewatch

CHAPTER 01

Data storage and movement

- The platform starts with storage.
- Data is stored once in open tables, brought in by pipelines and maintained automatically.
- Transactional and real-time data sit on the same storage.

IN THIS CHAPTER

Table formats

Ingestion and pipelines

Transactional and real-time data

Table maintenance

Table formats

Keep one consistent copy of your data, in an open format.

CHALLENGES

- Batch and streaming data often sit in separate stores, sometimes in closed or complex formats.
- A job that fails halfway can leave a table inconsistent.
- A schema change can break the jobs that read it.

Delta Lake is now the single format for batch and streaming alike.

- Databricks stores the data once, in **open tables** in your cloud storage.
- Delta adds **transactions** to those files.
- Batch jobs and streaming jobs write to the same tables without corrupting them.
- The schema can change without breaking readers.
- Tables can also use **Apache Iceberg**, the other open table format.
- **Multi-table transactions** commit changes to several tables at once.

Lakehouse / Delta Lake GA

Delta Lake provides open ACID transactions for both batch and streaming data workloads.

↗ lakehouse

Ingestion and pipelines

Connectors and pipelines are managed by the platform.

The data then has to get into this storage.

CHALLENGES

- Getting it there means writing connector code for each source and wiring job dependencies by hand.

Lakeflow now hands both jobs over to the platform.

- It brings the data in and transforms it.
- Lakeflow Connect provides **managed connectors** to databases and SaaS applications, query-based and community connectors, and Auto Loader for files.
- Zerobus Ingest brings in **event data**.
- **Spark Declarative Pipelines** describe the transformations, and Lakeflow works out the order in which to run them.
- They are written in the Lakeflow Pipelines Editor, or on a **visual canvas** in Lakeflow Designer, and can run in **real-time mode**.
- Lakeflow Jobs **orchestrates** the workflows.

Lakeflow GA

Lakeflow provides data ingestion, orchestration, and transformation components for building data pipelines.

↗ data engineering

Transactional and real-time data

Applications and agents keep their data on the same platform as the analytics, under the same governance.

The analytical data is now in place. Applications and agents also read and write transactional data, and that data usually lives in a separate database with its own copy.

CHALLENGES

- It is not governed there.
- Feeding analytics from it takes replication pipelines and a second system to maintain.
- The copies lag behind the source.

Lakebase now keeps that data next to the analytics, under the same rules.

- It is a **serverless Postgres** database inside the platform, for the state of applications and agents.
- **Synced tables** bring lakehouse data into Lakebase, and **Lakehouse Sync** writes Lakebase tables back to the lakehouse.
- **Lakehouse//RT** answers SQL queries in **under 100 milliseconds** directly on lakehouse tables, so dashboards and applications no longer need a separate serving copy.
- LTAP is the architecture these capabilities deliver: one dataset, written in Lakebase and read in Lakehouse//RT, with **no replication pipeline** between them.

Lakebase — serverless Postgres GA

Lakebase provides serverless PostgreSQL storage for agent state and application data with Unity Catalog integration.

↗ projects

Lakehouse//RT BETA

Lakehouse//RT provides serverless SQL query execution for real-time analytics on Delta and Iceberg tables.

↗ introducing lakehousert real...

LTAP ANNOUNCED

LTAP is an architecture that Lakebase capabilities deliver. It combines transactional processing and analytics in a unified lakehouse platform.

↗ databricks launches ltap first...

Table maintenance

Tables are maintained automatically, with no jobs to schedule.

The tables then need maintenance.

CHALLENGES

- Someone has to schedule it and tune it by hand.

Predictive Optimization now maintains the tables on its own.

- It decides when to compact files, remove old versions and cluster the data, based on how it is queried.
- It runs on **serverless compute**, and each decision is recorded for audit.

Predictive Optimization GA

Predictive Optimization automatically tunes Delta tables on Unity Catalog using ROI-based optimization.

↗ predictive optimization

CHAPTER 02

Unity Catalog

- Unity Catalog governs everything stored on the platform.
- Access, policies, quality, cost, business definitions and sharing are set in one place.
- Machine learning, agents and Genie all build on it.

IN THIS CHAPTER

Catalog and access

Policy and classification

Observability

Cost and usage

Semantic definitions

Data sharing

Catalog and access

Everything is now in one place. Access and lineage then have to be handled once for all of it.

- **Unity Catalog** applies **one set of permissions**, lineage and audit to tables and files.
- The catalog organizes every asset, and **Discover** lets users find them.
- Lineage follows the data across the platform, and **external lineage** extends it to systems outside Databricks.
- Access is granted **by role** or **by attribute**.
- Users can **request access** to an asset.
- Other engines access the same tables through **catalog commits** and the **Unity REST API**.
- **Secrets** are stored and governed in the catalog.

Catalog Explorer & AI-assisted discovery

Enables AI-powered discovery of tables, models, and functions with lineage tracking and data quality indicators.

↗ eda tutorial ↗ sql ↗ what is lakeflow designer ↗ catalog explorer ↗ databricks utils

Policy and classification

Access policies depend on knowing what each table holds.

- **Data classification** tags **sensitive data** such as PII.
- **Governed tags** carry those labels, and **tag propagation** applies them to the data derived from a tagged table.
- The tags feed **row- and column-level access policies**.
- **Governance Hub** shows the state of governance in one dashboard.

Classification / profiling / monitoring GA

Provides AI-driven data classification and quality monitoring integrated with Unity Catalog.

↗ unity catalog

Observability

Get alerted when a table goes stale, and repair it as an audited job.

CHALLENGES

- Data freshness is invisible and nobody is notified, so a stale table can go unnoticed.
- Repairing it is manual and risky.

Data Quality Monitoring now puts quality checks where access is controlled.

- **Profiles** track metrics over time and detect **drift** in tables and in predictions.
- Anomaly detection flags **stale or broken tables** and can trigger a repair job.
- **Health indicators** show the quality and freshness of each table in the catalog.
- Alerts go out by email, Slack or webhook.

Anomaly detection & remediation PUP

Detects anomalies in data with automated remediation capabilities that are still emerging.

↗ unity catalog

Monitoring & evaluation GA

Data/model quality monitoring, drift detection, and evaluation.

↗ data quality monitoring ↗ eval monitor
↗ monitor diagnose endpoints

Cost and usage

- **System tables** hold the platform's **billing and usage data**.
- They are queried with SQL like any other table.
- Audit logs and the history of automatic table maintenance are recorded there too.

Semantic definitions

An answer is only trusted if "revenue" or "active customer" means the same thing as in the official reports.

- **Metric views** define governed, reusable metrics in Unity Catalog, computed at query time.
- **Domains** group assets by business domain.
- **Pages** describe business concepts.

Data sharing

- **OpenSharing** is the protocol for sharing data.
- **SecureConnect** shares data that sits behind a firewall.
- **Clean Rooms** let organizations work on shared data without exposing their raw data to each other.
- **Marketplace** lists data, models and applications from other providers.

CHAPTER 03

Data science and machine learning

- Data scientists work on the governed data in notebooks, with an AI assistant.
- They build features, train on classic or serverless compute, track the work in MLflow, serve the models and automate their deployment.

IN THIS CHAPTER

Workspace

Feature engineering in Unity Catalog

Training compute

MLflow 3 (ML lifecycle management)

Model Serving (ML models)

Deployment automation

Workspace

- Data scientists work in notebooks that run Python, SQL, R and Scala directly on the **governed tables**.
- Several people can edit the same notebook at once, and notebooks are **versioned in Git**.
- Exploration uses the tools the team already knows: pandas, Spark, SQL and built-in data profiles.
- **Genie Code** is the AI assistant inside notebooks, the SQL editor, pipelines and dashboards.
- It **writes and explains code** from plain-language requests.
- It works in a pane next to the notebook or on a **full page**.
- A **command-line version**, in beta, brings it into the terminal and into CI pipelines.
- It also runs **scheduled tasks**, including as a task in Lakeflow Jobs.

Notebooks & collaboration GA

Collaborative multi-language notebooks, Git folders, and in-notebook assistance.

↗ notebooks ↗ notebooks collaborate ↗ genie code ↗ repos

Data prep & EDA GA

Exploration and preparation of data for modeling.

↗ eda tutorial ↗ sql ↗ what is lakeflow designer ↗ catalog explorer ↗ databricks utils

DS languages & frameworks GA

Languages and frameworks for data science include Python, R, Spark and Ray.

↗ release notes runtime 19ml ↗ databricks runtime ml ↗ ray

Genie Code GA

Genie Code is an AI assistant for notebooks and SQL with pay-per-use pricing.

↗ genie code ↗ release notes 2026 october ↗ genie code autonomous ai coding...

Feature engineering in Unity Catalog

Build features once & reuse them across teams and models.

Exploration ends in features, the inputs a model learns from. A feature computed one way for training and another way in production gives wrong predictions.

CHALLENGES

- Teams duplicate the same features across projects.
- Nobody sees where each one comes from or who may use it.

Feature Engineering in Unity Catalog ends the duplication with one shared registry.

- It keeps each definition in one place, with the same governance as the data.
- Teams write features to **feature tables**, or declare them as **Feature Views**, which compute them automatically.
- Stream Feature Views compute them from **streaming data**.
- Training sets are built with **point-in-time correct joins**, and the same definitions feed predictions in production.
- For real-time use, features are served from an **online store** on Lakebase.
- **Feature serving** returns them over REST, and models look them up automatically at inference time.

Feature Engineering in Unity Catalog GA

Feature Engineering in Unity Catalog manages governed features for training and serving, with an online store on Lakebase.

↗ feature store

Training compute

Train at any scale, with GPUs on demand.

Some datasets and models do not fit on one machine.

CHALLENGES

- Distributed training code is hard to write correctly.
- GPUs are scarce.
- Clusters configured by hand are error-prone or sit idle.

Databricks Runtime for Machine Learning takes much of that work off the team.

- It runs training on **classic clusters**, with managed libraries such as TorchDistributor, DeepSpeed and Ray.
- AI Runtime provides **GPUs on demand** with no cluster to manage, and it scales to zero when the job ends.
- The **AI Runtime SDK** spreads training across GPUs, and the **AI Runtime CLI** submits it from the command line.

Distributed training & GPU GA

Scale training across GPUs and clusters.

↗ databricks runtime ml ↗ ai runtime ↗ quickstart
↗ distributed training ↗ spark pytorch distributor

AI Runtime — serverless GPU PUP

Serverless GPU access for fine-tuning, distributed training, and batch inference with A10 and H100 instances.

↗ ai runtime

MLflow 3 (ML lifecycle management)

Repeat any experiment, and know which version runs in production.

With features in place, the team trains many models and compares them. Comparing them requires that every result can be reproduced.

CHALLENGES

- Each team keeps its metrics to itself.
- Nothing tracks model versions over time.
- Promotions are manual and error-prone.

MLflow handles the bookkeeping.

- It records every training run with its parameters, metrics, code version and artifacts, so any result can be **reproduced and compared**.
- Every model in production is registered in Unity Catalog with its versions, its aliases such as champion and challenger, and its **lineage** back to the training data.
- Promoting a new version means **moving an alias**, and model update triggers or deployment jobs can start the next steps automatically.

MLflow GA

Experiment tracking, autologging, and model management.

↗ mlflow ↗ tracking ↗ eval monitor ↗ overview

Models in Unity Catalog GA

Manages model versions, aliases for production stages, and Unity Catalog lineage.

↗ mlflow

Model Serving (ML models)

Put any model into production on serverless infrastructure.

A registered version then has to serve traffic.

CHALLENGES

- Each kind of model needs its own serving system, provisioned by hand, with access control split across them.
- Idle machines keep costing money.

Model Serving replaces them all as a single serverless service.

- Its endpoints run the registered version, for real-time requests and for **batch scoring**.
- **Custom model endpoints** serve models built with scikit-learn, XGBoost, PyTorch or Hugging Face.
- Endpoints scale with demand, **down to zero**.
- They handle more than **25,000 queries per second** with low added latency.
- Features are looked up from the online store **at serving time**.
- **Endpoint telemetry** monitors the usage of each endpoint.
- **Inference tables** log **every request and response**.

Model Serving GA

Model Serving provides serverless endpoints with pay-per-token, provisioned throughput, and batch billing.

↗ model serving ↗ foundation model overview

Model Serving & batch inference GA

Real-time and batch inference on governed models.

↗ model serving ↗ model inference

Inference Tables

Inference Tables capture AI Gateway request and response payloads to Unity Catalog Delta tables for quality analysis.

↗ inference tables beta ↗ inference tables

Deployment automation

Each change then has to reach production safely and the same way every time.

- **Declarative Automation Bundles** describe code, jobs, models and their environments as **files in Git**.
- The **Databricks CLI** deploys them from a terminal or a CI pipeline.
- Each change goes from development to production through that **automated path**.

Declarative Automation Bundles (DABs)

Declarative Automation Bundles define ML infrastructure as code in YAML for reproducible deployments across environments.

↗ bundles ↗ settings

CHAPTER 04

Agent development

- An agent combines models with company data and tools.
- Teams choose the models, retrieve knowledge, connect tools, then build, deploy and give memory to the agent.
- Business users build no-code agents with Genie Agents.

IN THIS CHAPTER

Model access

Custom LLMs

Batch inference: AI Functions

Knowledge retrieval

Tools

Agent build

Agent deployment

Agent memory

No-code agents: Genie Agents

Model access

Each task can get the best model, from any of the main providers.

CHALLENGES

- Depending on one provider creates lock-in.
- Token spend grows unpredictably.
- Models from different providers are tested piece by piece.

Foundation Model APIs remove the dependency on any single vendor.

- They give access through **one API** to models from providers that include Anthropic, OpenAI, Google and Meta.
- They are billed **per token**, with a **priority** option, or with **provisioned throughput**.
- The catalog includes **OpenJev**, the Databricks judge model.
- Because applications call the same API for every model, moving to a newer model stays a **small change**.
- Models hosted at a provider are registered as **external models**, with their keys **stored centrally**.
- **Unity Gateway** is the single access layer in front of all of these models.

Foundation Model APIs GA

Foundation Model APIs include Anthropic Claude, OpenAI GPT, and Google Gemini with pay-per-token and provisioned throughput pricing.

↗ foundation model apis ↗ supported models

External model services GA

Databricks governs access to externally hosted models through Unity Catalog and the Gateway.

↗ foundation model overview ↗ model serving

Unity Gateway GA

Unity Gateway controls all AI traffic for models, agents, and custom tools.

↗ ai governance ↗ ai gateway governance layer...

Custom LLMs

Adapt a model to your data with less GPU memory, paid by the minute.

Sometimes a general model does not know the company's vocabulary or tasks well enough.

CHALLENGES

- Full fine-tuning needs large amounts of GPU memory and is expensive and slow.
- Once tuned, the model often sits apart from the systems that serve the others.

AI Runtime now runs this work with lighter methods, and keeps the result on the platform.

- It adapts an open model to company data with LoRA, QLoRA or full fine-tuning, on **serverless GPUs**.
- The tuned model is logged in MLflow and served on **Model Serving** like any other model.

Fine-tuning (on AI Runtime) PUP

Fine-tuning runs on AI Runtime, with QLoRA, LoRA or full fine-tuning on serverless GPUs. Foundation Model Fine-tuning was removed on Aug 14, 2026.

↗ ai runtime

AI Runtime — serverless GPU PUP

Serverless GPU access for fine-tuning, distributed training, and batch inference with A10 and H100 instances.

↗ ai runtime

Batch inference: AI Functions

Apply AI to large volumes of data and documents in place.

CHALLENGES

- Running a model over company data usually means sending it to an outside service, building separate pipelines for documents and splitting the work into batches by hand.
- What comes back is unstructured.

AI Functions now run this work where the information already lives.

- They call a model **from SQL** or from a pipeline to classify, extract or summarize data and documents at scale.
- This needs no agent.
- ai_query calls any model, ai_classify classifies, ai_extract extracts fields, ai_parse_document parses documents and ai_transcribe transcribes audio.
- Classification and extraction also have a **no-code interface**.
- The results come back with a **fixed schema**, and the data stays in Unity Catalog.
- Applications call the same functions through a **REST API**.

AI Functions GA

SQL- & pipeline-native AI for batch transformation of data and documents.

↗ ai functions ↗ introducing aidecide make fast...

Knowledge retrieval

Retrieve targeted data from documents, based on checked sources.

An agent answers questions as they come. To answer well, it first has to find the right information.

CHALLENGES

- Search indexes are built and kept in sync by hand.
- They often match on vectors alone.

AI Search fixes this by grounding answers in the right material.

- ai_prep_search first splits the documents into chunks and prepares them for it.
- AI Search indexes documents and tables for **vector, hybrid and full-text search**, keeps the index in sync with the Delta tables, and returns **citations**.
- The **ai_search** function queries the index from SQL.
- Agents can also use **Genie Agents** as a source of context.

AI Search (ex-Vector Search) GA

AI Search combines vector and keyword search with reranking, Delta sync, and citation tracking.

↗ ai search

Tools

Agents reach tools through one governed route.

Answering from documents only reads data. Many tasks also need actions, such as running a query, opening a ticket or changing code. Each of these tools needs a connection and permissions.

CHALLENGES

- Teams write a custom connector for every one of them and host the servers themselves.
- Credentials pile up across agents.
- Tool use stays hidden.

The **Model Context Protocol (MCP)** lets every connection be governed in one place instead.

- It is the **standard way** for an agent to call tools.
- Databricks runs **managed MCP servers** for its own services, such as Genie, AI Search and SQL.
- Managed MCP connectors give **governed access** to Google Workspace, Microsoft 365, Slack and GitHub.
- Teams can register their own MCP servers or host them as Databricks Apps.
- **Unity Catalog functions** and **skills** serve as tools as well.
- Every tool is a **Unity Catalog object**, so permissions and audit work the same way as for data.
- Databricks Sandbox runs the code an agent writes in an **isolated environment**, with scoped access to data.

Managed MCP servers BETA

Managed MCP servers provide governed agent access to Databricks services including analytics, search, and SQL.

↗ managed mcp ↗ release notes 2026 september

External connectors GA

External connectors provide governed agent access to cloud services including Google Workspace, Microsoft 365, and GitHub.

↗ external sources ↗ introducing mcp connectors...

MCP Services + custom MCP GA

MCP Services lets you register external MCPs as governed assets and deploy custom MCPs as Databricks Apps.

↗ mcp services ↗ use mcp in agents

Databricks Sandbox BETA

Isolated serverless code execution, scoped UC access.

↗ sandbox ↗ deploy

Agent build

Build the agent you need, with any framework and any model.

An agent combines retrieval and tools with its own logic.

CHALLENGES

- Platforms often impose the framework and the model.

Custom agents lift these limits.

- They are written in code with frameworks such as LangGraph, LangChain or the OpenAI SDK.
- They can use **any model** and any MCP tool.
- **AI Bridge** packages connect these frameworks to Databricks: databricks-openai, databricks-langchain, databricks-dspy and databricks-mcp.
- The **Agent Bricks CLI** creates a new project from templates.
- Omnigent can also run them as a **meta-harness**.

Custom Agents GA

Build agents using code-first frameworks like LangGraph or CrewAI with agent.toml configuration.

↗ build agents ↗ agent bricks dais 2026

Agent deployment

Once built, an agent has to run in production.

- **AgentKit** is a **durable agent server**, shipped in the databricks-agentbricks SDK package.
- Agents run on **Agent Runtime**, on Databricks Apps.
- Applications query a deployed agent through the **Invocation API**, the **Responses API** or **SQL**.

Agent memory

Agents remember what users told them.

CHALLENGES

- Agents often start every conversation from nothing.
- Users then repeat their instructions each time.

Managed memory keeps that knowledge for them.

- It is stored in **Lakebase**, and an agent searches it to **remember context** across conversations.
- **Sessions** keep the state of each conversation.
- Memory can also be built with **memory pipelines**.
- Several agents can **share** the same memory.

Managed memory & sessions BETA

Lakebase-backed, cross-session, semantic search (Sep 16).

↗ release notes 2026 september ↗ managed memory

No-code agents: Genie Agents

- Domain experts build agents **without code** in Genie Agents.
- They are part of **AI for business users**.

CHAPTER 05

Observability and evaluation

- Before an agent reaches users, its behavior has to be visible and its answers measured.
- MLflow traces each request, evaluates the answers and versions the prompts.
- Runtime telemetry covers what runs in production.

IN THIS CHAPTER

MLflow Tracing

MLflow evaluation

Prompt Registry (prompt management)

Runtime telemetry

MLflow Tracing

Trace request details and cost, step by step.

CHALLENGES

- Agent chains are opaque to the people who run them.
- When an answer is wrong, the failing step is hard to isolate.
- The cost of each request is unknown.

MLflow makes every step visible.

- It traces every request through the agent.
- It records each model call, tool call and retrieval step, with inputs, outputs, **latency and tokens**, and it works with more than 30 agent frameworks.
- When an answer is wrong, the trace shows **the step where it went wrong**.
- The traces are stored in **Unity Catalog**.

MLflow 3 for GenAI GA

MLflow provides end-to-end tracing for 30+ frameworks, LLM-based evaluation, and production monitoring.

↗ genai ↗ overview ↗ eval monitor

MLflow evaluation

Improve the quality of new agent versions.

A trace shows one request. Judging quality needs a score across many requests.

CHALLENGES

- Without such scores, traces are reviewed by hand and versions cannot be compared.
- Expert labels sit in separate spreadsheets.

MLflow evaluation measures the answers.

- It applies **LLM judges** and custom **scorers** to traces.
- It runs in batch before a release and continuously on **live traffic**.
- Each agent version gets its own evaluation run, so versions can be compared.
- The **Review App** lets domain experts test the agent in a chat interface and **label its answers**.
- The labels attach to the traces and become **evaluation datasets**.
- They are also used to align the judges with the experts.

Agent evaluation & monitoring GA

Collects execution traces, applies LLM judges, and displays evaluation dashboards.

↗ genai ↗ eval monitor ↗ label existing traces

Review Apps GA

Review Apps enable human-in-the-loop labeling of traces and chat interactions for eval datasets.

↗ label existing traces

Prompt Registry (prompt management)

Know which prompt each version of an agent uses.

CHALLENGES

- Prompts change with no record.
- Staging and production drift apart.

Prompt Registry keeps a version of every prompt.

- Staging and production aliases point to them, so a change is tracked **like a code change**.
- Each prompt is stored with its **model parameters**, such as the temperature.
- Unity Catalog permissions control who can use each prompt.

MLflow Prompt Registry GA

Versioned prompts, staging/prod aliases.

↗ prompt registry

Runtime telemetry

In production, each endpoint and each app also has to be watched.

- Model Serving reports **telemetry** for each endpoint.
- Databricks Apps export their **telemetry** to Unity Catalog, with agent traces in MLflow.
- **Inference tables** store the requests and responses in Delta tables for quality analysis.
- **System tables** record tokens, latency and errors for audit and troubleshooting.

Inference Tables

Inference Tables capture AI Gateway request and response payloads to Unity Catalog Delta tables for quality analysis.

↗ inference tables beta ↗ inference tables

CHAPTER 06

AI governance and security

- Unity Catalog and Unity Gateway govern AI with the same rules as data.
- The catalog holds the AI assets and their permissions.
- The gateway applies policies, monitoring and cost controls to every call, including the calls of coding agents.

IN THIS CHAPTER

Unity Catalog for AI assets

Unity Gateway: policies

Unity Gateway: monitoring

Unity Gateway: cost and capacity

Unity Gateway: coding agent governance

Security and compliance (Databricks AI Security Framework)

Unity Catalog for AI assets

Each model, agent and tool can be traced to the data behind it and to everyone who used it.

CHALLENGES

- Models, agents and tools are not governed as assets of their own.
- Permissions are enforced differently across them.
- External credentials are scattered.
- Nobody knows which data feeds which model.

Unity Catalog extends the existing rules to these assets.

- It governs models, agents, tools and credentials in the **same way as tables and files**.
- These assets include models, functions, hosted model services, external model provider services, memory stores, skills and traces.
- Each of them is a Unity Catalog object with permissions, lineage and an audit trail.
- Lineage follows the chain **from data to model to agent**.
- Credentials for external services are stored in the catalog and **encrypted**.
- Permissions also cover the use of **foundation models** and **AI Functions**, and **ABAC GRANT policies** apply to models and services.

Unity Catalog for AI assets GA

Unity Catalog for AI assets provides role-based access control, lineage tracking, and audit trails for models and agents.

↗ unity catalog

Unity Gateway: policies

Every model and tool call follows the same policies.

The catalog sets who may use each asset. Each model call and each tool call also needs control at the moment it happens.

CHALLENGES

- Prompts and responses go through without guardrails.

Unity Gateway puts one control plane in charge.

- It sits in front of all model and tool traffic and is built on Unity Catalog.
- **Service policies** set the rules for the calls to each service.
- **LLM-as-a-judge policies** check prompts and responses with a model.
- **Sensitive data** is redacted, and **partner guardrails** can be added.
- **Request tags** attribute each call to a user, team or project.

Unity Gateway GA

Unity Gateway controls all AI traffic for models, agents, and custom tools.

↗ ai governance ↗ ai gateway governance layer...

Unity Gateway: monitoring

See who calls what, and keep a record of every call.

CHALLENGES

- Teams connect to providers on their own, so nobody sees who uses which model.
- Nothing is logged for an audit.

Unity Gateway records the traffic that passes through it.

- **Inference tables** log every request and response.
- Usage is tracked **per user and team**, and a **usage dashboard** shows it.
- A **unified trace table** collects the traces in one place.

Unity Gateway GA

Unity Gateway controls all AI traffic for models, agents, and custom tools.

↗ ai governance ↗ ai gateway governance layer...

Unity Gateway: cost and capacity

Keep AI costs within budget.

CHALLENGES

- Spend on models grows unchecked.

Unity Gateway puts limits on it, centrally.

- **Budgets** and **cost caps** apply per user, team or project.
- **Rate limits** cap the number of requests.
- **Smart routing** picks the model for each request and falls back to another one when a provider is unavailable.

Unity Gateway GA

Unity Gateway controls all AI traffic for models, agents, and custom tools.

↗ ai governance ↗ ai gateway governance layer...

Unity Gateway: coding agent governance

- Coding agents connect to Unity Gateway with the **ug CLI**.
- Their configuration is managed **centrally**.
- Their model calls then follow the same policies, monitoring and cost controls as the rest of the AI traffic.

Unity Gateway GA

Unity Gateway controls all AI traffic for models, agents, and custom tools.

↗ ai governance ↗ ai gateway governance layer...

Security and compliance (Databricks AI Security Framework)

Permissions and guardrails address known risks. A security team also needs the full list of risks and the control that covers each one.

- The Databricks AI Security Framework lists the risks of AI systems and the controls that address them.
- Version 3.0 covers 97 risks and 73 controls, including 35 risks **specific to agents**.
- The controls include **least-privilege tool access** in Unity Catalog, sandboxed code execution, guardrails in Unity Gateway, and human review.
- **Databricks Geos** keep data in a chosen geography, for **data residency**.
- The **compliance security profile** turns on additional security controls for regulated workloads.

DASF v3.0 GA

Databricks AI Security Framework v3.0 catalogs 97 risks with 73 controls, including 35 agentic-specific risks.

↗ agentic ai security new risks...

CHAPTER 07

AI for business users

- Business users reach the governed data and the agents without writing code.
- Genie One answers their questions in plain language, and Genie Agents carry the knowledge of each business area.
- Genie also works inside the tools they already use, and applications run next to the data.

IN THIS CHAPTER

Genie One (conversational assistant)

Genie Agents (no-code agents)

Genie Ontology (business context layer)

Genie integrations

AI/BI Dashboards

Applications

Genie ZeroOps (autonomous pipeline operations)

Genie budgets (per-user limits)

Genie One (conversational assistant)

Anyone in the business can question the data directly.

CHALLENGES

- Most people who need data do not write SQL.
- Their questions queue up at the data team.
- The answer often comes after the decision.

Genie One gives everyone a single entry point to the data.

- It lets business users ask questions in **plain language**.
- They get answers from governed company data, with citations and with **their own permissions** applied.
- When a dashboard or an agent already answers the question, Genie One sends the user there.
- It runs on the **web**, as a desktop app on macOS, and on **iOS** and **Android**.
- Other AI assistants reach it through the **Genie One MCP server**.
- **Web search** adds outside sources to its answers.
- Users with **account-only access** can use it too.

Genie One GA

Genie One provides cross-asset BI assistance with memory and MCP server access for named users.

↗ genie one ↗ introducing genie one genie...
↗ release notes 2026

Genie Agents (no-code agents)

The knowledge of each business area becomes available to everyone in it.

Once the definitions are shared, domain experts can package what they know for others.

CHALLENGES

- Today that knowledge stays with a few people.
- Dashboards only answer the questions they were built for.
- Deeper analysis is spread across separate tools.

Genie Agents spread that expertise instead.

- They let a domain expert build a **no-code assistant** for a business area.
- The assistant can follow up on questions, run **multi-step analyses** and explain its reasoning, and it can be embedded in other applications.

Genie Agents GA

Genie Agents enable no-code domain-specific assistants with limits up to 50 sources and 200,000 conversations.

↗ genie one ↗ release notes 2026

Genie Ontology (business context layer)

Each metric has one certified definition, used everywhere.

An answer is only trusted if "revenue" or "active customer" means the same thing as in the official reports.

CHALLENGES

- Metrics are defined differently across dashboards, queries and teams.
- Business context is scattered in documents, chats and tickets.
- An AI can then pick an obsolete or draft definition.

Genie Ontology grounds answers in the certified version.

- It holds those definitions.
- Each KPI is **defined once**, as a metric view in Unity Catalog.
- The ontology also learns **business context** from existing dashboards, queries and pipelines.
- It ranks sources by certification, usage and freshness, so Genie answers from **certified definitions first**.

Genie Ontology PUP

Learned business-context layer grounding Genie answers.

↗ introducing genie one genie...

Genie integrations

Many users spend their day in chat tools and spreadsheets.

- Genie answers questions inside **Microsoft Teams** and **Slack**.
- It also works in **Excel** and **Google Sheets**.
- It is available in **Microsoft Copilot Cowork** as well.

AI/BI Dashboards

Analysts and business people serve themselves, without waiting for a specialist.

CHALLENGES

- Building a dashboard takes SQL skills, so requests wait.

AI/BI Dashboards come with an assistant.

- A dashboard can be built from a **plain-language request**.
- The assistant finds the tables, writes the queries and picks the charts, and the author **approves each step**.

AI/BI Dashboards (Lakeview) GA

AI-assisted authoring, dashboard-level Genie, metric views.

↗ dashboards ↗ release notes 2026

Applications

Data and AI applications run on the platform itself, next to the governed data.

When the same question comes back every week, it is worth turning into a small application.

CHALLENGES

- Usually each one runs on separate infrastructure, with its own identity management and secrets.
- It sits apart from the governed data.
- Access control and audit trails are hard to put in place.

Databricks Apps now hosts them itself.

- Developers build applications with the **frameworks they already use**, such as Streamlit, Gradio, React or FastAPI, and deploy them.
- **Databricks AppKit** is a framework for building them.
- The apps run **serverless** on the platform.
- They sign users in with the platform's identity and read data through Unity Catalog with **each user's permissions**.
- Custom MCP servers run the same way.
- App Spaces give admins **one place to govern** who deploys and uses which app.
- **Genie App Builder** generates an app from a plain-language description, within the **limits the admins set**.

Databricks Apps + App Spaces GA

Databricks Apps hosts interactive data and AI apps on Databricks, governed by Unity Catalog.

↗ databricks apps ↗ introducing databricks apps and...

Genie App Builder BETA

Genie App Builder generates Databricks Apps from natural language descriptions with governance.

↗ genie app builder ↗ genie app builder deploy apps...

Genie ZeroOps (autonomous pipeline operations)

- **Genie ZeroOps** runs **pipeline operations** on its own.
- Its agent investigates a problem, proposes a fix in a sandbox, and waits for **human approval**.
- It is announced.

Genie budgets (per-user limits)

- **Genie budgets** set **per-user limits** on Genie usage.

CHAPTER 08

Developer tools

- Developers bring their own coding agents to the platform and migrate existing SQL code.

IN THIS CHAPTER

Omnigent (coding agent harness)

Agentic code converter (SQL migration)

Omnigent (coding agent harness)

Coding agents share context and follow central policies.

CHALLENGES

- Each coding-agent tool is its own isolated world, so content is copied between them by hand.

Omnigent puts them behind one interface.

- It works with coding agents such as **Claude, Codex or Cursor**.
- Agents are defined in **portable YAML**, for local or managed execution.
- Developers can **share a live session**.
- The managed version runs code in Databricks Sandbox.

Omnigent BETA

Omnigent unifies agent development across Claude, Codex, and Cursor with managed infrastructure and policies.

↗ omnigent.ai ↗ omnigent

Agentic code converter (SQL migration)

- The **agentic code converter** converts SQL written for other systems into Databricks SQL.

CHAPTER 09

Vertical applications

- Databricks builds complete applications for specific domains on the same platform.
- Both are in private preview.

IN THIS CHAPTER

CustomerLake (customer data platform)

Lakewatch (agentic SIEM)

CustomerLake (customer data platform)

Campaigns and personalization run on governed customer profiles that stay current.

CHALLENGES

- Each system holds its own part of the customer, often duplicated and without lineage.
- Audiences are built and activated by hand.

CustomerLake now covers this work as a ready-made application.

- In private preview, it builds **customer 360 profiles** on the lakehouse.
- It joins transactions, behavior and engagement into one profile per customer.
- AI-assisted **identity resolution** merges duplicate records.
- Campaign agents create and **activate audiences**, and a real-time profile API feeds personalization.

CustomerLake PPR

CustomerLake delivers agentic capabilities for customer profiles, identity resolution, and campaign management.

↗ introducing customerlake...

Lakewatch (agentic SIEM)

Keep every log and investigate faster.

CustomerLake applies the platform to customer data. The same pattern applies to security data.

CHALLENGES

- Telemetry is split across separate tools.
- Threat hunting and triage are manual.
- Detection rules are hard to version and test.

Lakewatch now lets agents share the load.

- In private preview, it runs security operations on the lakehouse.
- It keeps security, IT and business logs in one place at **object-storage cost**, so they no longer have to be deleted early.
- Autonomous agents **investigate alerts** and help write detection rules, which are versioned and tested like code.

Lakewatch PPR

Lakewatch provides agentic security information and event management with Panther integration.

↗ databricks enters security...

Capability map by section

01 DATA STORAGE AND MOVEMENT

1.1 Table formats
- Delta Lake
- Apache Iceberg
- Multi-table transactions

1.2 Ingestion and pipelines
- Lakeflow Connect (ingestion connectors)
  Managed connectors · Query-based connectors · Community connectors · Auto Loader (file ingestion)
- Zerobus Ingest (event ingestion)
- Spark Declarative Pipelines (pipeline framework)
  Lakeflow Pipelines Editor (authoring UI) · Real-time mode
- Lakeflow Designer (visual pipeline builder)
- Lakeflow Jobs (orchestration)

1.3 Transactional and real-time data
- Lakebase (transactional database): LTAP architecture
  Synced tables (lakehouse to Lakebase) · Lakehouse Sync (Lakebase to lakehouse)
- Lakehouse//RT (real-time analytics)

1.4 Table maintenance
- Predictive Optimization

02 UNITY CATALOG

2.1 Catalog and access
- Catalog
- Discover (asset discovery)
- Lineage
  External lineage
- Role-based access control
- Attribute-based access control
- Access requests
- External engine access
  Catalog commits · Unity REST API
- Secrets

2.2 Policy and classification
- Data classification
- Governed tags
  Tag propagation
- Governance Hub (governance dashboard)

2.3 Observability
- Data Quality Monitoring
  Anomaly detection · Health indicators

2.4 Cost and usage
- System tables (billing and usage data)

2.5 Semantic definitions
- Metrics (metric views)
- Domains (business domains)
- Pages (business concept pages)

2.6 Data sharing
- OpenSharing (sharing protocol)
  SecureConnect (sharing behind a firewall)
- Clean Rooms
- Marketplace

03 DATA SCIENCE AND MACHINE LEARNING

3.1 Workspace
- Notebooks
- Genie Code (coding assistant)
  Genie Code pane (in notebooks) · Full-page Genie Code · Genie Code CLI · Scheduled tasks · Lakeflow Jobs task

3.2 Feature engineering in Unity Catalog
- Feature tables
- Feature Views
  Stream Feature Views
- Online Feature Stores
- Feature serving

3.3 Training compute
- Databricks Runtime for Machine Learning (classic compute)
- AI Runtime (serverless GPU compute)
  Distributed training (AI Runtime SDK) · AI Runtime CLI

3.4 MLflow 3 (ML lifecycle management)
- Experiment tracking
- Models in Unity Catalog (model registry)

3.5 Model Serving (ML models)
- Custom model endpoints
- Endpoint telemetry
- Inference tables

3.6 Deployment automation
- Declarative Automation Bundles
- Databricks CLI

Capability map by section

04 AGENT DEVELOPMENT

4.1 Model access

- Foundation Model APIs (hosted models)
  Pay-per-token · Priority pay-per-token · Provisioned throughput · OpenJev (Databricks judge model)
- External model providers
- Unity Gateway (model access layer)

4.2 Custom LLMs

- Fine-tuning (on AI Runtime)
- Custom LLM serving (on Model Serving)

4.3 Batch inference: AI Functions

- ai_query (general inference)
- ai_classify (classification)
  Classification UI
- ai_extract (information extraction)
  Information Extraction UI
- ai_parse_document (document parsing)
- ai_transcribe (audio transcription)
- AI Functions REST API

4.4 Knowledge retrieval

- ai_prep_search (chunking and preparation)
- AI Search (vector, hybrid and full-text search)
- ai_search (search SQL function)
- Genie Agents as a context source (see AI for business users)

4.5 Tools

- Managed MCP servers
- Managed MCP connectors
- External MCP servers
- Unity Catalog functions
- Skills
- Databricks Sandbox (code execution tool)

4.6 Agent build

- Custom agents (code-first agents, any framework)
- AI Bridge (framework integration packages)
  databricks-openai · databricks-langchain · databricks-dspy · databricks-mcp
- Agent Bricks CLI
  CLI project templates
- Omnigent as meta-harness (see Developer tools)

4.7 Agent deployment

- AgentKit (durable agent server)
  databricks-agentbricks (SDK package)
- Agent Runtime (on Databricks Apps)
- Agent query interfaces
  Invocation API · Responses API · SQL

4.8 Agent memory

- Managed memory
- Sessions
- Memory pipelines

4.9 No-code agents: Genie Agents

05 OBSERVABILITY AND EVALUATION

5.1 MLflow Tracing

- Traces in Unity Catalog

5.2 MLflow evaluation

- LLM judges
- Scorers
- Review App (human labeling)

5.3 Prompt Registry (prompt management)

5.4 Runtime telemetry

- Endpoint telemetry (Model Serving)
- Databricks Apps telemetry
- Inference tables
- System tables

Capability map by section

06 AI GOVERNANCE AND SECURITY

6.1 Unity Catalog for AI assets
- Models
- Functions
- Model services (hosted model endpoints)
- Model provider services (external providers)
- Memory stores
- Skills
- Traces
- Foundation model permissions
- AI Functions permissions
- ABAC GRANT policies for models and services

6.2 Unity Gateway: policies
- Service policies
- LLM-as-a-judge policies
- Sensitive data redaction
- Partner guardrails
- Request tags (attribution)

6.3 Unity Gateway: monitoring
- Inference tables (request and response logging)
- Usage tracking
- Usage dashboard
- Unified trace table

6.4 Unity Gateway: cost and capacity
- Budgets
- Cost caps
- Rate limits
- Smart routing

6.5 Unity Gateway: coding agent governance
- ug CLI (coding agent connection)
- Centralized coding agent configuration

6.6 Security and compliance (Databricks AI Security Framework)
- Databricks Geos (data residency)
- Compliance security profile

07 AI FOR BUSINESS USERS

7.1 Genie One (conversational assistant)
- Web app
- Desktop app (macOS)
- iOS app
- Android app
- Genie One MCP server
- Web search
- Account-only user access

7.2 Genie Agents (no-code agents)

7.3 Genie Ontology (business context layer)

7.4 Genie integrations
- Microsoft Teams
- Slack
- Excel
- Google Sheets
- Microsoft Copilot Cowork

7.5 AI/BI Dashboards

7.6 Applications
- Databricks Apps (application hosting)
  Databricks AppKit (app framework) · App Spaces
- Genie App Builder (no-code app builder)

7.7 Genie ZeroOps (autonomous pipeline operations)

7.8 Genie budgets (per-user limits)

08 DEVELOPER TOOLS

8.1 Omnigent (coding agent harness)

8.2 Agentic code converter (SQL migration)

09 VERTICAL APPLICATIONS

9.1 CustomerLake (customer data platform)

9.2 Lakewatch (agentic SIEM)